On-call Runbook Copilot

An incident-response web app: alerts become incidents, and a copilot proves each root cause from the service's own logs, live, or says unknown.

Ashutosh · Technical documentation · October 2026

Live gallery: ashu-stark.github.io/oncall-copilot/ · GitHub: github.com/Ashu-Stark/oncall-copilot

Contents

1. Overview

2. The web app

3. Incidents, alerts and the data model

4. Live investigations

5. Runbook library

6. The problem

7. Architecture

8. How an incident is investigated

9. Retrieval in detail

10. The runbooks

11. Fault injection

12. Evaluation

13. LLM investigator (optional)

14. Security

15. Testing and delivery

16. Safety and design decisions

17. Limitations

18. Running it

19. Interview questions

1 Overview

The On-call Runbook Copilot is an incident-response web app for the engineer on call. Alerts become incidents. For each one, the copilot finds the runbook that matches the symptoms, **checks that runbook's claims against the service's own logs**, and reports the root cause, the log lines that prove it, and the next steps, streaming every step to the browser as it happens. If no runbook is confirmed by the logs, it answers **UNKNOWN** and escalates instead of guessing. Engineers acknowledge, assign, take notes, edit runbooks and get a postmortem draft built from the incident record.

Stack: React + TypeScript (Vite) · FastAPI · PostgreSQL with SQLAlchemy 2 and Alembic · Server-Sent Events · JWT + bcrypt · Docker Compose · GitHub Actions · Render blueprint.

0% false alarms on healthy traffic	**100%** correct root cause on detected incidents	**3 → 0** wrong diagnoses, without → with log verification	**55** faults injected into a real API

It runs against a real system: the platform API of **HOI (Hospitality Operations Intelligence)**, a multi-tenant FastAPI + MongoDB service. HOI's code is not modified; faults are injected from outside by swapping dependencies HOI already isolates, so the copilot reads exactly the log lines HOI writes in production.

On-call Runbook Copilot | Incidents | Dashboard | Runbooks | Asha Rao (lead) | Sign out

Incidents / #8

SEV1 · critical | Triggered | simulated: db_down

HighErrorRate + ReadinessFailing: 67% of 60 requests returned 5xx; readiness probe failed 4 times

SERVICE	OPENED	TIME TO ACKNOWLEDGE	TIME TO RESOLVE	LOG WINDOW
hoi-platform	25 Oct, 10:39 pm	–	–	101 lines

Assignee: Unassigned | Acknowledge | Resolve (false alarm) | Postmortem draft

Copilot investigation

Re-run

Alert rules on the log window: HighErrorRate, ReadinessFailing

HighErrorRate 67% of 60 requests returned 5xx

ReadinessFailing readiness probe failed 4 times

Built a search query from the alerts and the most frequent errors

```
HighErrorRate: 67% of 60 requests returned 5xx | ReadinessFailing: readiness probe failed 4 times | unhandled error rid=<rid> POST /public/guest/<token>/chat | pymongo.errors.ServerSelectionTimeoutError: hoi-cluster-shard-00-01.example.mongodb.net:27017: timed…
```

Retrieved 5 candidate runbooks (hybrid search)

1. **RB01** rrf 0.03279 (bm25 #1, dense #1) via RB01#symptoms
2. **RB13** rrf 0.03226 (bm25 #2, dense #2) via RB13#mitigation
3. **RB08** rrf 0.03175 (bm25 #3, dense #3) via RB08#symptoms
4. **RB02** rrf 0.03055 (bm25 #7, dense #4) via RB02#symptoms
5. **RB15** rrf 0.03033 (bm25 #4, dense #8) via RB15#diagnosis

RB01: 3/3 checks passed, confirmed

CHECK	EXPECTED	FOUND IN LOGS	
`search_logs(pattern="ServerSelectionTimeoutError").count`	>= 1	36	✓ pass
`status_count(status="5xx").share`	>= 0.100	0.667	✓ pass
`health().ready`	== false	false	✓ pass

▸ Evidence the checks used (6 log lines)

Root cause: RB01 · Database unreachable (MongoDB)

RB01 · Database unreachable (MongoDB) SEV1

Root cause: the API cannot reach the MongoDB cluster, so every request that reads or writes data fails.

Verified: 3/3 runbook checks passed on this incident's logs. Grounding check: every cited log line exists.

```
#23 INFO hoi.platform: rid=3d496a469f6a GET /ready -> 503 1ms
#24 ERROR hoi.platform: unhandled error rid=03e0dcc94f8f GET /inventory/stock | pymongo.errors.ServerSelectionTimeoutError: hoi-cluster-shard-00-01.example.mongodb.net:27017: timed out (configured timeouts: socketTimeoutMS: 20000.0ms, connectTimeoutMS: 20000.0ms)
#25 INFO hoi.platform: rid=03e0dcc94f8f GET /inventory/stock -> 500 6ms
#26 ERROR hoi.platform: unhandled error rid=c678cadcb328 GET /dispatch/jobs | pymongo.errors.ServerSelectionTimeoutError: hoi-cluster-shard-00-01.example.mongodb.net:27017: timed out (configured timeouts: socketTimeoutMS: 20000.0ms, connectTimeoutMS: 20000.0ms)
```

▸ Runbook steps (7)

Simulation ground truth: RB01 ✓ matches

hybrid+checks · investigation 30 ms incl. saving each step (shown slowly on purpose)

Alerts

HighErrorRate + ReadinessFailing ×1
67% of 60 requests returned 5xx; readiness probe failed 4 times
from simulator (lead.demo)

Timeline

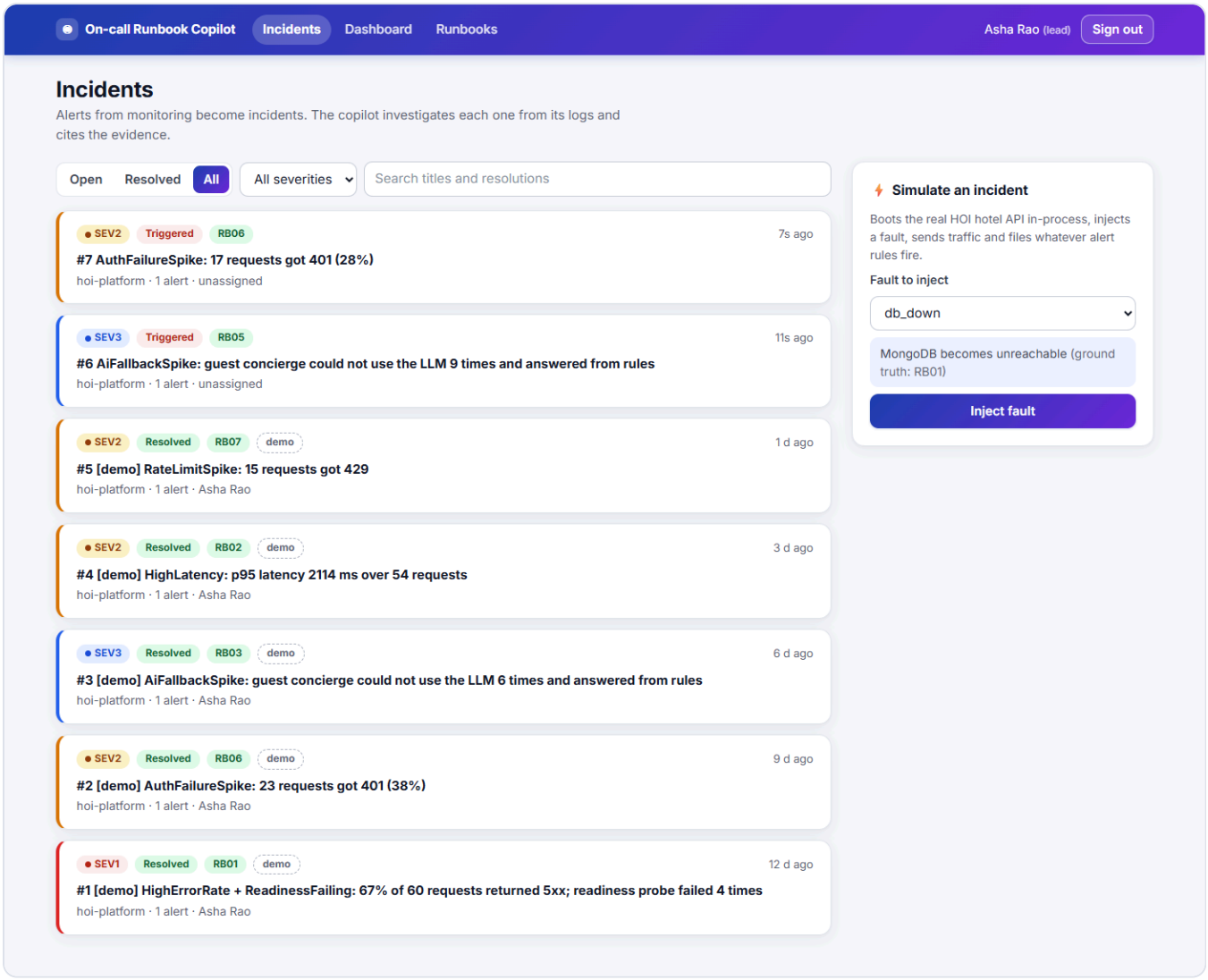
- HighErrorRate + ReadinessFailing: 67% of 60 requests returned 5xx; readiness probe failed 4 times
  10:39:50 pm · simulator (lead.demo)
- Copilot investigation started
  10:39:50 pm · copilot (auto)
- Root cause: RB01 · Database unreachable (MongoDB)
  10:39:51 pm · copilot

Add a note

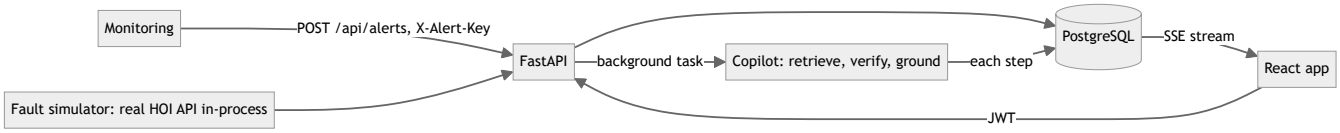
What did you check? What did you change?

Add note

2 The web app



Page	What it does
Incidents	Inbox filtered by status, severity and text; severity, status and verdict pills; refreshes every 10 s. A simulator panel injects one of the evaluation's faults into the real HOI API and opens whatever incident the alert rules produce.
Incident	Facts (service, opened, time to acknowledge/resolve, log window), workflow buttons, assignee; the copilot's investigation streaming live; alerts; timeline with notes.
Runbooks	Library and editor with live validation (line-numbered errors), parsed checks preview, version history; only leads can save.
Dashboard	Open incidents by severity, median MTTA/MTTR, incidents by service and per day, verdict outcomes, simulated incidents whose verdict matched the injected fault, and the offline evaluation table.
Postmortem	Draft built from the record: summary, impact, root cause with evidence, timeline, resolution, action items; download as Markdown.



On-call Runbook Copilot | Incidents | Dashboard | Runbooks | Asha Rao (lead) | Sign out

Dashboard

Last 30 days: 8 incidents (5 are demo incidents with made-up response times).

3	5 min	38 min	8/8
open incidents	median time to acknowledge (n=6)	median time to resolve (n=5)	simulated incidents where the verdict matched the injected fault

Open by severity

SEV3 1
SEV2 1
SEV1 1

Incidents by service

hoi-platform 8

Copilot verdicts

confirmed runbook 8

Incidents per day

2026-10-13 1
2026-10-16 1
2026-10-19 1
2026-10-22 1
2026-10-24 1
2026-10-25 3

Offline evaluation (eval/run_eval.py)

55 scenarios on the real HOI API (45 injected incidents). Detection 91% · false alarms 0% · retrieval recall@1 93%, recall@3 98%.

MODE	ACCURACY (ALL)	ON DETECTED INCIDENTS	WRONG ROOT CAUSE	ABSTAINED
full (the copilot)	**93%**	**100%**	**0**	**0**
hybrid-top1	87%	93%	3	0
bm25-top1	87%	93%	3	0
dense-top1	87%	93%	3	0
bm25+checks	93%	100%	0	0

Optimistic by construction: the same person wrote the faults, the alert rules and the runbook checks.

3 Incidents, alerts and the data model

Table	Holds
`incidents`	Title, service, severity, status, assignee, created/acknowledged/resolved times, captured health probe, simulation ground truth, demo flag, resolution
`alerts`	Fingerprint, name, summary, severity, source, labels, count, first/last seen, and `open_fingerprint` (unique)
`log_lines`	The incident's log window; `seq` 1..n is the id the copilot cites
`investigations`, `investigation_steps`	Status, verdict, full diagnosis, runbook versions used; one row per step
`incident_events`	Timeline: alerts, repeats, status changes, assignment, notes, verdicts, severity changes
`runbooks`, `runbook_versions`	Current markdown and every saved version with editor and comment
`users`, `services`	bcrypt password hash and role; service owners

Deduplication that holds under concurrency

An alert whose fingerprint matches an **open** incident only increments a counter and adds a timeline entry, so nobody is paged twice. `open_fingerprint` equals the fingerprint while the incident is open and becomes NULL when it is resolved; its unique constraint means two identical alerts arriving at the same moment cannot both open an incident: the second insert fails and is folded into the first. A test fires 8 identical alerts in parallel and expects exactly one incident with count 8.

Workflow

triggered → acknowledged → investigating → resolved. A false alarm can be resolved straight from triggered or acknowledged; resolved is final, and resolving requires a resolution text. Invalid moves return 409 with the allowed ones. MTTA and MTTR are the mean and median minutes from creation to acknowledgement and to resolution.

4 Live investigations

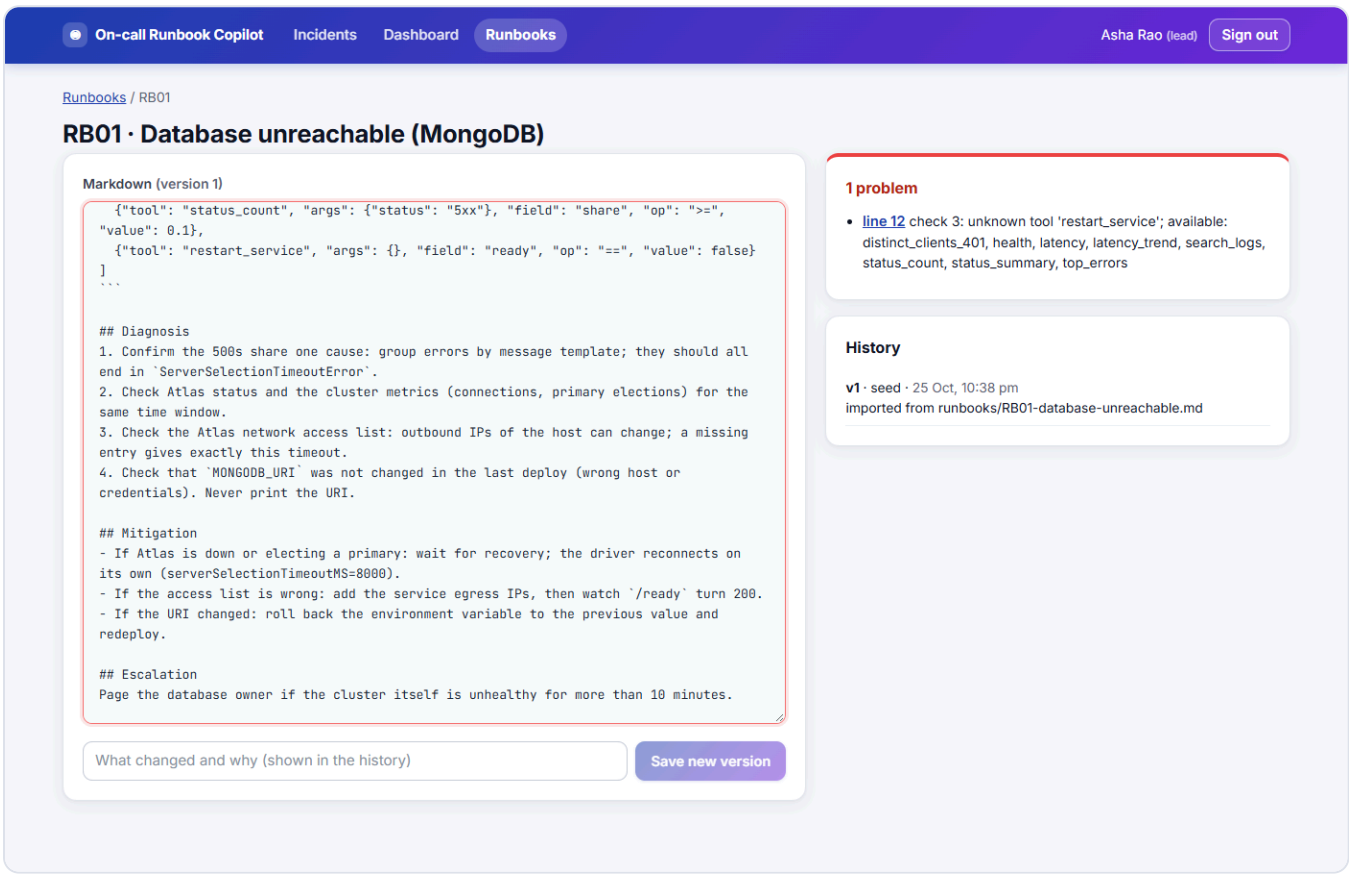
The copilot's `investigate()` gained one optional argument, an observer called after each step (alert rules, query, retrieval, each runbook's checks). It only observes, so the diagnosis is unchanged. The web app runs it as a background task:

1. The incident's log lines are loaded back into the copilot's own log store in order, so the ids it cites are the incident's line numbers.
2. The readiness probe captured with the alert is replayed to the `health` tool (or the probe's failure, if the service was not running).
3. Each step is committed to `investigation_steps`, with the cited log lines attached to each check.
4. The browser reads `GET /api/investigations/{id}/stream`: Server-Sent Events, one `step` event per step (id = step number) and a final `done`.

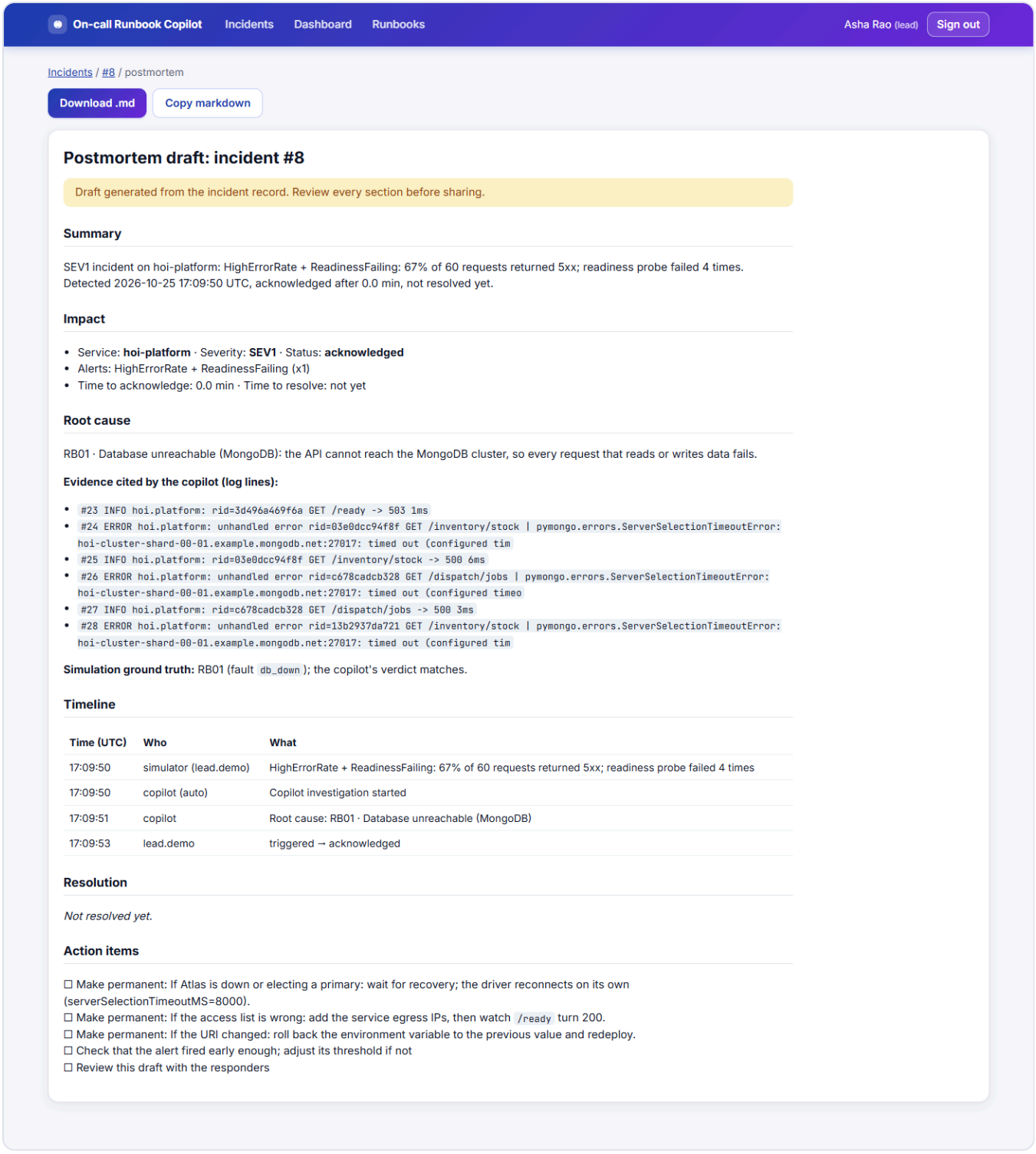
The stream is read with `fetch` and a small parser rather than `EventSource`, so the session token goes in the `Authorization` header and never in a URL; after a dropped connection the client reconnects with `Last-Event-ID` and the server resumes after that step. Steps are paced (350 ms by default) so people can follow them; the copilot's own time is reported separately.

> **Equivalence test.** For all 9 faults, the copilot is run on the live HOI instance and on the stored, replayed log window; the verdicts and cited log ids are identical.

5 Runbook library



Runbooks live in Postgres in the same markdown format as `runbooks/*.md` and are parsed by the copilot's own parser, so what saves is exactly what the investigator reads. Validation reports invalid JSON in the checks block (with its line number), unknown tools (the list comes from the read-only `Tools` class, so a "restart_service" check is impossible), arguments that do not fit the tool's signature, unknown operators, missing sections and an id change. Saving creates a new version; the editor sends the version it started from and gets 409 if someone saved in between. The retriever is rebuilt only when a runbook changes, and each investigation records which versions it used.



6 The problem

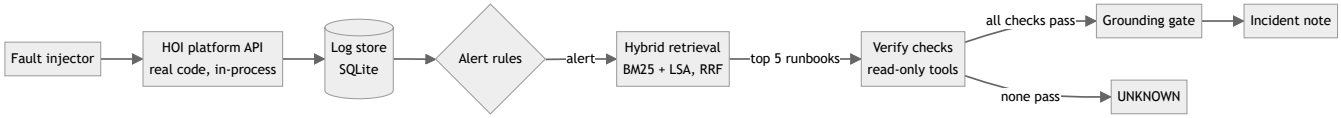
At 3 a.m. an alert says "38% of requests are failing". The engineer must work out *why*: is the database down, did a deploy break configuration, is someone attacking the login endpoint? Teams write **runbooks** ("if you see X, check A, B and C; the cause is usually D"), but under pressure people pick the wrong runbook, or trust a symptom that several different causes share.

The hard cases are **look-alike incidents**:

Symptom	Possible causes that look the same	What separates them
High latency	Slow database · cold start after idle	Is the slowness sustained, or only in the first requests?
Spike of HTTP 401	Signing secret rotated · password guessing · normal token expiry	Are the 401s spread over many endpoints, concentrated on `/auth/login`, or a trickle?
Spike of HTTP 429	Rate limit set too low · brute-force attack	Which endpoints get 429: guest chat across rooms, or the login endpoint?
AI answers degrade	Provider rate limit · missing API key · model output changed	The exact fallback reason in the logs

A pure search or LLM approach picks by *wording* and confuses these. The copilot picks by *evidence*.

7 Architecture



Component	File	Responsibility
Log store	`copilot/logstore.py`	Captures Python log records into SQLite; parses HOI's access lines (`rid=… GET /rooms -> 500 4ms`) into status, path and latency columns.
Tools	`copilot/tools.py`	Read-only queries: status counts, latency percentiles and trend, 401 spread, log search, top error templates, health probe.
Alert rules	`copilot/alerts.py`	Decide *what* looks wrong (error rate, readiness, latency, 401/429 spikes, AI fallbacks, dependency errors, startup failure).
Runbooks	`runbooks/*.md` , `copilot/runbooks.py`	16 Markdown runbooks with a machine-checkable `checks` block; split into section chunks for retrieval.
Retrieval	`copilot/retrieval.py`	BM25 written out by hand, LSA vectors (TF-IDF + truncated SVD), Reciprocal Rank Fusion.
Investigator	`copilot/agent.py`	Builds the query, verifies candidate runbooks, applies the grounding gate, renders the note.
LLM investigator	`copilot/llm_agent.py`	Optional: an LLM chooses the tools via function calling; same grounding gate.
Harness	`harness/`	Runs HOI in-process on an in-memory database, generates traffic, injects faults.
Evaluation	`eval/run_eval.py`	55 scenarios, five modes (ablation), metrics and misses.
Demo site	`site/build_site.py`	Runs every fault and publishes the incident gallery to GitHub Pages.

8 How an incident is investigated

1. Alert

Alert rules run over the log store. Each alert carries a summary and the ids of the log rows that triggered it. Thresholds are assumptions for a small service (tens of requests per window), for example *5xx share ≥ 10%* or *p95 latency ≥ 400 ms*.

2. Query

The investigator writes the query an engineer would type: the alert names and summaries, the three most frequent warning/error *templates* (log lines with ids, tokens and numbers replaced, so identical messages group together), and the most common AI fallback reason.

3. Retrieve

The query is ranked against runbook sections by two methods, which are fused by rank:

- **BM25**: exact words such as `ServerSelectionTimeoutError` or `429`. The tokenizer also splits CamelCase error names into words.
- **LSA vectors**: TF-IDF reduced to 48 dimensions with truncated SVD, which captures words that occur together ("slow", "latency", "p95").

4. Verify

For each of the top 5 runbooks, in rank order, the investigator runs the runbook's checks. A check names a tool, its arguments, a field, an operator and a value:

```
{"tool": "status_count", "args": {"status": "5xx"}, "field": "share", "op": ">=", "value": 0.1}
```

The first runbook whose checks **all** pass is the diagnosis. Look-alike runbooks contain checks that rule each other out (for example, "slow database" requires `slow_only_at_start == false`; "cold start" requires it to be true).

5. Ground and report

Before anything is shown, a grounding gate checks that the runbook exists and that every cited log id exists in the store. The note lists the alerts, the diagnosis and severity, the root cause, up to six cited log lines and the runbook's diagnosis and mitigation steps.

9 Retrieval in detail

BM25

For a query term *t* and a section *d*:

```
score(t, d) = idf(t) · f · (k1 + 1) / (f + k1 · (1 − b + b · |d| / avgdl))
idf(t)      = ln(1 + (N − df + 0.5) / (df + 0.5))
```

f is how often *t* occurs in *d*. **k1 = 1.5** limits how much repeating a word helps; **b = 0.75** corrects for long sections; the **+1** inside the logarithm keeps very common words from getting a negative weight.

Reciprocal Rank Fusion

```
rrf(runbook) = 1 / (60 + rank_bm25) + 1 / (60 + rank_dense)
```

RRF uses only ranks, so the two score scales (BM25 sums and cosine similarities) never need to be calibrated against each other. A runbook's score is that of its best section.

10 The runbooks

Id	Runbook	Severity	Injected in evaluation?
RB01	Database unreachable	SEV1	yes
RB02	Database slow	SEV2	yes
RB03	LLM provider rate limited (429)	SEV3	yes
RB04	LLM API key missing	SEV3	yes
RB05	LLM output no longer matches schema	SEV3	yes
RB06	Token signing secret changed	SEV2	yes
RB07	Password guessing on login	SEV2	yes
RB08	Tourism portal dependency down	SEV3/4	as a no-impact case
RB09	Unsafe production configuration	SEV1	yes
RB10	Rate limit set too low	SEV2	yes
RB11–RB16	Cold start, token expiry, inventory compensation, dispatch conflicts, CORS, memory pressure	various	no: near-miss decoys

11 Fault injection

Each fault swaps one dependency HOI already isolates and is undone afterwards, so scenarios cannot leak into each other. The investigation runs **while the fault is still active**, so health probes see the broken system as on a real on-call.

Fault	How it is injected	What HOI logs
Database down	MongoDB accessors raise `ServerSelectionTimeoutError`	500s with that exception; readiness failures
Database slow	A proxy sleeps before every collection call	Latency in seconds, all 200s
LLM rate limited	Chat adapter raises "HTTP 429"	`concierge fallback to rules: llm call failed (HTTP 429)`
LLM key missing	Keys removed from the environment	`… set LLM_API_KEY or GROQ_API_KEY`
Schema drift	Model returns prose or another JSON shape	`… invalid JSON for the schema`
Secret rotated	New `HOI_JWT_SECRET` after tokens were issued	401s across many endpoints
Password guessing	Many failed logins from one IP	401 then 429 on `/auth/login`
Rate limit too low	Guest chat limit set to 1–8 per minute	429 on guest chat
Unsafe prod config	Production mode without secrets	Startup failure (HOI refuses to boot)
Portal down	Places API times out or returns 503	Nothing: HOI serves cached places (correct answer: no incident)

12 Evaluation

55 scenarios: 11 types × 5 variants with different seeds, fault strengths and background noise (typo logins, 404s). Five modes form an ablation, to show what each part contributes.

Mode	Correct (all 55)	Correct (41 detected)	Wrong root cause
Hybrid retrieval + log-verified checks (the copilot)	93%	100%	0
Hybrid retrieval, top-1, no verification	87%	93%	3
BM25 only, top-1	87%	93%	3
LSA vectors only, top-1	87%	93%	3
BM25 + verification	93%	100%	0

- **Verification removes the wrong answers** (3 → 0): retrieval by wording confuses the look-alike pairs.
- **Dense retrieval adds nothing here**: 16 short runbooks full of exact error names favour keywords. It should matter with paraphrased alerts or many more runbooks.
- **All 4 misses are detection misses**, not wrong diagnoses: 8 and 12 password guesses stay under the login limit, and chat limits of 5 or 8 per minute were not hit by the test traffic.
- **No false alarms** on 10 scenarios with nothing wrong, including the portal outage that HOI absorbs.

> **Honest caveat.** The same person wrote the faults, the alert rules and the runbook checks, so these numbers are optimistic. They show the pipeline works end to end on a real codebase and how its parts compare; they are not an accuracy estimate for unseen production incidents.

13 LLM investigator (optional)

`copilot/llm_agent.py` lets an LLM choose the tools through function calling (any OpenAI-compatible API: Groq, OpenAI or a local Ollama). Its final answer must be JSON with a runbook id and cited log ids, and it passes the same grounding gate plus one more rule: **cited log ids must have been returned by a tool during this investigation**, so the model cannot invent evidence. It is implemented but not yet evaluated, so no accuracy is claimed for it.

14 Security

Concern	How
Passwords	bcrypt (cost 12); unknown users cost the same time as wrong passwords
Sessions	JWT (HS256, 8 h); production refuses to start without a 32+ character secret
Roles	engineer: incidents, notes, simulator; lead: also runbook changes
Brute force	5 failed logins per user per minute, then 429
Alert ingestion	`X-Alert-Key` compared in constant time (or a signed-in engineer)
Browser	Token only in the Authorization header (also for SSE); postmortems render React elements, never raw HTML
LLM	Optional, server-side only; the HOI harness removes LLM keys from the system under test

15 Testing and delivery

Suite	Count	Covers
Copilot unit tests	11	Log parsing, BM25, runbook parsing, checks, grounding gate, abstention
Web app (real Postgres)	37	Auth, roles, throttle, dedup incl. 8 parallel alerts, workflow, MTTA/MTTR, stored steps, abstention, SSE stream and resume, postmortem, runbook validation, versions and conflicts, a new runbook changing the next verdict, simulator, live-vs-replayed equivalence for 9 faults
UI (Testing Library + axe-core)	15	SSE parser and reconnect, live investigation panel, abstention display, highlighted evidence, inbox filters, simulator flow, runbook editor errors and conflict, read-only for engineers, markdown safety, accessibility checks

Backend tests use `TEST_DATABASE_URL` when set (CI: a Postgres service container) or start an embedded Postgres with `pgserver`; the schema is created with the Alembic migrations, as in production. Delivery: an

API image that migrates on start and clones HOI at a pinned commit for the simulator, an nginx image that serves the app and proxies `/api` with buffering off for SSE, `docker-compose.yml`, and a Render blueprint.

16 Safety and design decisions

- **Read-only tools.** The copilot never restarts, rolls back or changes anything; a human applies the fix.
- **Abstain rather than guess.** A wrong root cause at 3 a.m. sends people down the wrong path; UNKNOWN escalates.
- **Evidence over wording.** Retrieval only proposes; checks against the logs decide.
- **No secrets in logs.** HOI never logs the database URI or guest messages; the copilot only reads what HOI writes.

17 Limitations

- Faults are single and clean; real incidents overlap (a slow database that then times out).
- About 60 requests per scenario from one process; thresholds are set for that scale.
- LSA is learned from 16 runbooks; a pretrained embedding model would generalise better and can be plugged in through `Embedder`.
- All HOI data is synthetic demo data.
- One API process: simulations run one at a time (the harness changes process-wide state) and the SSE stream polls Postgres every 250 ms. Several instances would need `LISTEN/NOTIFY` or a queue and a separate worker.
- Alerts carry their log window; a real deployment would pull logs from the log platform.
- The 5 demo incidents are real simulations with made-up, backdated response times, labelled demo; the dashboard's MTTA/MTTR include them.

18 Running it

```
pip install -r backend/requirements.txt
$env:DATABASE_URL = python backend/devdb.py         # embedded Postgres (or: docker compose up --
build)
alembic -c backend/alembic.ini upgrade head
python -m backend.app.seed --demo-incidents
uvicorn backend.app.main:app --port 8104
cd frontend; npm install; npm run dev               # http://localhost:5184

python -m pytest -q tests                            # 11 copilot tests
python -m pytest -q backend/tests                    # 37 web-app tests (real Postgres)
cd frontend; npm test                               # 15 UI tests
python eval/run_eval.py                             # 55 scenarios (offline)
python demo.py db_down                              # one incident note on the command line
```

19 Interview questions

Why not just ask an LLM what is wrong?

It would answer from wording and could invent causes. Here every diagnosis must be confirmed by checks on the real logs and cite log lines that exist; otherwise the answer is UNKNOWN.

What does BM25's k1 do? And b?

k1 caps the benefit of repeating a word (term-frequency saturation); b scales how much long documents are penalised.

Why Reciprocal Rank Fusion instead of adding scores?

BM25 scores and cosine similarities are on different scales; RRF only uses ranks, so no calibration is needed.

How do you know it works?

55 injected faults on a real API, five-mode ablation, zero false alarms, and verification taking wrong diagnoses from 3 to 0. And I state that the numbers are optimistic because I wrote the faults and the checks.

How do you stop two identical alerts from opening two incidents?

A unique column that holds the fingerprint only while the incident is open. Two concurrent inserts cannot both succeed; the loser folds its alert into the winner. A "check, then insert" in Python would race.

Why fetch instead of EventSource for the live stream?

EventSource cannot send an Authorization header, and tokens in URLs end up in logs. fetch with a small parser keeps the header and still supports Last-Event-ID resume.

How do you know the web app gives the same verdict as the evaluated copilot?

It calls the same function on the same log lines, in the same order, with the same health result; a test compares live and replayed verdicts and cited ids for all 9 faults.

What would you do next?

Evaluate the LLM mode, add overlapping faults, use a pretrained embedding model, and test on runbooks and incidents written by someone else.